corresponding multivariate Gaussian distribution.

> **NOTE** **Zero mean versus constant mean**
>
> **Instead of the `gpytorch.means.ZeroMean` class in our implementation from Section 2.4.2, we are using the `gpytorch.means.ConstantMean` class to initialize our mean function. However, this constant mean function has a default value of zero, so effectively, we are still implementing the same GP model.**
>
> **While these two choices lead to identical models for now, we will see how `gpytorch.means.ConstantMean` allows us to adjust the constant mean value to obtain a better model shortly.**

Let's now initialize an object of this class, train it on our training data, and visualize its predictions. We do this with the following code:

```
# declare the GP
lengthscale = 1
noise = 1e-4

likelihood = gpytorch.likelihoods.GaussianLikelihood()
model = ConstantMeanGPModel(train_x, train_y, likelihood)

# fix the hyperparameters
model.covar_module.lengthscale = lengthscale
model.likelihood.noise = noise

model.eval()
likelihood.eval()

visualize_gp_belief(model, likelihood)
```

Here, we initialize the GP model and set its hyperparameters—the length scale and the noise variance—to be 1 and 0.0001, respectively. We will see how to set the values of these hyperparameters appropriately later in this chapter; for now, let's just stick with these values. Finally, we call the helper function we just wrote `visualize_gp_belief()` on our GP model, which produces Figure 3.5.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>